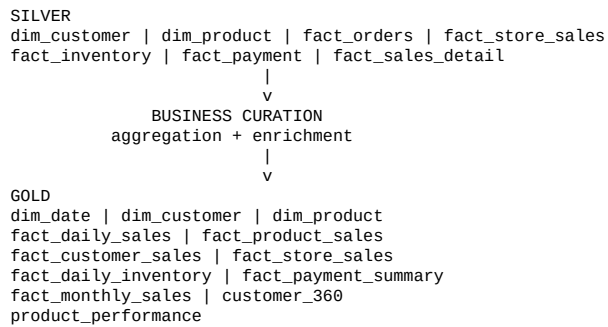


Retail Analytics — Gold Layer

Silver → Gold Analytics Pipeline | Databricks PySpark

Complete reference notebook for transforming curated Silver data into business-ready Gold dimensions, facts, aggregates, Customer 360 and Product Performance tables.

Gold Layer Flow



Complete PySpark Code

Copy this code into a Databricks Python notebook. The implementation uses Delta tables and creates the Gold schema automatically.

```
# =====
# RETAIL ANALYTICS - SILVER -> GOLD PIPELINE
# Databricks PySpark Notebook
# =====

from pyspark.sql import functions as F
CATALOG = "retail_catalog"
SILVER_SCHEMA = "silver_retail"
GOLD_SCHEMA = "gold_retail"
SILVER = f"{CATALOG}.{SILVER_SCHEMA}"
GOLD = f"{CATALOG}.{GOLD_SCHEMA}"

# 1. CREATE GOLD SCHEMA
spark.sql(f"CREATE SCHEMA IF NOT EXISTS {GOLD}")

# 2. READ SILVER TABLES
df_customer = spark.table(f"{SILVER}.dim_customer")
df_product = spark.table(f"{SILVER}.dim_product")
df_orders = spark.table(f"{SILVER}.fact_orders")
df_store_sales = spark.table(f"{SILVER}.fact_store_sales")
df_inventory = spark.table(f"{SILVER}.fact_inventory")
df_payment = spark.table(f"{SILVER}.fact_payment")
df_sales_detail = spark.table(f"{SILVER}.fact_sales_detail")

# 3. GOLD DIM_DATE
r = (df_sales_detail
    .select(F.min("transaction_date").alias("min_date"),
            F.max("transaction_date").alias("max_date"))
    .first())
min_date = r["min_date"]
max_date = r["max_date"]
if min_date is None:
    min_date = spark.sql("SELECT current_date()").first()[0]
if max_date is None:
    max_date = min_date
df_date = (
    spark.sql(f"""
        SELECT explode(sequence(
            to_date('{min_date}'),
            to_date('{max_date}'),
            interval 1 day
        )) AS calendar_date
    """)
    .withColumn("date_key", F.date_format("calendar_date", "yyyyMMdd").cast("int"))
    .withColumn("year", F.year("calendar_date"))
    .withColumn("quarter", F.quarter("calendar_date"))
    .withColumn("month", F.month("calendar_date"))
    .withColumn("month_name", F.date_format("calendar_date", "MMMM"))
    .withColumn("year_month", F.date_format("calendar_date", "yyyy-MM"))
    .withColumn("week_of_year", F.weekofyear("calendar_date"))
    .withColumn("day", F.dayofmonth("calendar_date"))
    .withColumn("day_of_week", F.dayofweek("calendar_date"))
    .withColumn("day_name", F.date_format("calendar_date", "EEEE"))
    .withColumn("is_weekend", F.dayofweek("calendar_date").isin(1,7))
)

# 4. GOLD DIM_CUSTOMER
df_gold_customer = (
    df_customer
    .withColumn(
        "customer_segment",
        F.when(F.col("is_active") == True, "ACTIVE").otherwise("INACTIVE")
    )
    .withColumn(
        "customer_lifecycle",
        F.when(F.col("customer_tenure_days") < 90, "NEW")
        .when(F.col("customer_tenure_days") < 365, "GROWING")
        .otherwise("LOYAL")
    )
    .withColumn(
        "customer_age_band",
        F.when(F.col("customer_tenure_days") < 30, "0-30 DAYS")
        .when(F.col("customer_tenure_days") < 90, "31-90 DAYS")
        .when(F.col("customer_tenure_days") < 365, "91-365 DAYS")
        .otherwise("1+ YEARS")
    )
)

# 5. GOLD DIM_PRODUCT
df_gold_product = (
    df_product
    .withColumn(
        "product_segment",
        F.when(F.col("unit_price") < 100, "VALUE")
        .when(F.col("unit_price") < 500, "STANDARD")
        .when(F.col("unit_price") < 1000, "PREMIUM")
        .otherwise("LUXURY")
    )
)

# 6. PREPARE SALES DETAIL
df_sales = (
    df_sales_detail
    .withColumn("gross_sales", F.coalesce(F.col("gross_product_value"),F.lit(0)))
    .withColumn("discount_amount", F.coalesce(F.col("discount_amount"),F.lit(0)))
    .withColumn("tax_amount", F.coalesce(F.col("tax_amount"),F.lit(0)))
    .withColumn("sales_amount", F.coalesce(F.col("sales_amount"),F.lit(0)))
    .withColumn("net_sales", F.col("sales_amount"))
    .withColumn("sales_year", F.year("transaction_date"))
    .withColumn("sales_month", F.month("transaction_date"))
    .withColumn("sales_quarter", F.quarter("transaction_date"))
    .withColumn("sales_year_month", F.date_format("transaction_date", "yyyy-MM"))
)
```

```

)
# 7. FACT DAILY SALES - grain: date + channel
df_daily_sales = (
    df_sales
    .groupBy("transaction_date", "sales_channel")
    .agg(
        F.countDistinct("transaction_id").alias("transaction_count"),
        F.sum("quantity").alias("units_sold"),
        F.sum("gross_sales").cast("decimal(18,2)").alias("gross_sales"),
        F.sum("discount_amount").cast("decimal(18,2)").alias("discount_amount"),
        F.sum("tax_amount").cast("decimal(18,2)").alias("tax_amount"),
        F.sum("net_sales").cast("decimal(18,2)").alias("net_sales"),
        F.avg("net_sales").cast("decimal(18,2)").alias("average_transaction_value")
    )
    .withColumn("year", F.year("transaction_date"))
    .withColumn("month", F.month("transaction_date"))
    .withColumn("quarter", F.quarter("transaction_date"))
    .withColumn("year_month", F.date_format("transaction_date", "yyyy-MM"))
    .withColumn("gold_load_timestamp", F.current_timestamp())
)
# 8. FACT PRODUCT SALES - grain: date + product
df_product_sales = (
    df_sales
    .groupBy("transaction_date", "product_id", "product_name",
             "category", "subcategory", "brand")
    .agg(
        F.countDistinct("transaction_id").alias("transaction_count"),
        F.sum("quantity").alias("units_sold"),
        F.sum("gross_sales").cast("decimal(18,2)").alias("gross_sales"),
        F.sum("discount_amount").cast("decimal(18,2)").alias("discount_amount"),
        F.sum("tax_amount").cast("decimal(18,2)").alias("tax_amount"),
        F.sum("net_sales").cast("decimal(18,2)").alias("net_sales")
    )
    .withColumn("year", F.year("transaction_date"))
    .withColumn("month", F.month("transaction_date"))
    .withColumn("year_month", F.date_format("transaction_date", "yyyy-MM"))
    .withColumn("gold_load_timestamp", F.current_timestamp())
)
# 9. FACT CUSTOMER SALES - grain: date + customer
df_customer_sales = (
    df_sales
    .groupBy("transaction_date", "customer_id", "customer_name", "customer_lifecycle")
    .agg(
        F.countDistinct("transaction_id").alias("transaction_count"),
        F.sum("quantity").alias("units_purchased"),
        F.sum("net_sales").cast("decimal(18,2)").alias("net_sales"),
        F.sum("discount_amount").cast("decimal(18,2)").alias("discount_amount"),
        F.sum("tax_amount").cast("decimal(18,2)").alias("tax_amount"),
        F.avg("net_sales").cast("decimal(18,2)").alias("average_order_value")
    )
    .withColumn("year", F.year("transaction_date"))
    .withColumn("month", F.month("transaction_date"))
    .withColumn("year_month", F.date_format("transaction_date", "yyyy-MM"))
    .withColumn("gold_load_timestamp", F.current_timestamp())
)
# 10. FACT STORE SALES - grain: date + store
df_gold_store_sales = (
    df_sales.filter(F.col("store_id").isNotNull())
    .groupBy("transaction_date", "store_id")
    .agg(
        F.countDistinct("transaction_id").alias("transaction_count"),
        F.sum("quantity").alias("units_sold"),
        F.sum("gross_sales").cast("decimal(18,2)").alias("gross_sales"),
        F.sum("discount_amount").cast("decimal(18,2)").alias("discount_amount"),
        F.sum("tax_amount").cast("decimal(18,2)").alias("tax_amount"),
        F.sum("net_sales").cast("decimal(18,2)").alias("net_sales"),
        F.avg("net_sales").cast("decimal(18,2)").alias("average_transaction_value")
    )
    .withColumn("year", F.year("transaction_date"))
    .withColumn("month", F.month("transaction_date"))
    .withColumn("year_month", F.date_format("transaction_date", "yyyy-MM"))
    .withColumn("gold_load_timestamp", F.current_timestamp())
)
# 11. FACT DAILY INVENTORY - grain: date + store + product
df_daily_inventory = (
    df_inventory
    .groupBy("inventory_date", "store_id", "product_id", "product_name",
             "category", "subcategory", "brand")
    .agg(
        F.max("available_qty").alias("available_qty"),
        F.max("reorder_level").alias("reorder_level"),
        F.max("safety_stock").alias("safety_stock"),
        F.max("inventory_value").cast("decimal(18,2)").alias("inventory_value"),
        F.max("stock_status").alias("stock_status"),
        F.max(F.when(F.col("reorder_required")==True, 1).otherwise(0))
        .alias("reorder_required")
    )
    .withColumn("year", F.year("inventory_date"))
    .withColumn("month", F.month("inventory_date"))
    .withColumn("year_month", F.date_format("inventory_date", "yyyy-MM"))
    .withColumn("gold_load_timestamp", F.current_timestamp())
)
# 12. FACT PAYMENT SUMMARY - grain: date + method + status
df_payment_summary = (
    df_payment
    .groupBy("payment_date", "payment_method", "payment_status")
    .agg(
        F.countDistinct("payment_id").alias("payment_count"),
        F.sum("payment_amount").cast("decimal(18,2)").alias("payment_amount"),
        F.sum(F.when(F.col("payment_success_flag")==True, 1).otherwise(0))
        .alias("successful_payment_count"),
        F.sum(F.when(F.col("payment_success_flag")==False, 1).otherwise(0))

```

```

        .alias("failed_payment_count")
    )
    .withColumn(
        "success_rate_pct",
        F.round(
            F.col("successful_payment_count") /
            F.when(F.col("payment_count")==0, None)
            .otherwise(F.col("payment_count")) * 100, 2
        )
    )
    .withColumn("year", F.year("payment_date"))
    .withColumn("month", F.month("payment_date"))
    .withColumn("year_month", F.date_format("payment_date", "yyyy-MM"))
    .withColumn("gold_load_timestamp", F.current_timestamp())
)
# 13. FACT MONTHLY SALES - grain: month + channel + category
df_monthly_sales = (
    df_sales
    .groupBy("sales_year", "sales_month", "sales_quarter", "sales_year_month",
            "sales_channel", "category")
    .agg(
        F.countDistinct("transaction_id").alias("transaction_count"),
        F.sum("quantity").alias("units_sold"),
        F.sum("gross_sales").cast("decimal(18,2)").alias("gross_sales"),
        F.sum("discount_amount").cast("decimal(18,2)").alias("discount_amount"),
        F.sum("tax_amount").cast("decimal(18,2)").alias("tax_amount"),
        F.sum("net_sales").cast("decimal(18,2)").alias("net_sales"),
        F.avg("net_sales").cast("decimal(18,2)").alias("average_transaction_value")
    )
    .withColumn("gold_load_timestamp", F.current_timestamp())
)
# 14. CUSTOMER 360 - grain: one row per customer
df_customer_metrics = (
    df_sales.groupBy("customer_id").agg(
        F.countDistinct("transaction_id").alias("total_transactions"),
        F.sum("quantity").alias("total_units_purchased"),
        F.sum("net_sales").cast("decimal(18,2)").alias("lifetime_sales"),
        F.sum("discount_amount").cast("decimal(18,2)").alias("lifetime_discount"),
        F.max("transaction_date").alias("last_purchase_date"),
        F.min("transaction_date").alias("first_purchase_date")
    )
)
df_customer_360 = (
    df_gold_customer.join(df_customer_metrics, "customer_id", "left")
    .withColumn("total_transactions", F.coalesce(F.col("total_transactions"), F.lit(0)))
    .withColumn("total_units_purchased", F.coalesce(F.col("total_units_purchased"), F.lit(0)))
    .withColumn("lifetime_sales", F.coalesce(F.col("lifetime_sales"), F.lit(0)).cast("decimal(18,2)"))
    .withColumn(
        "average_order_value",
        F.when(F.col("total_transactions")>0,
            F.col("lifetime_sales")/F.col("total_transactions"))
        .otherwise(0).cast("decimal(18,2)")
    )
    .withColumn(
        "days_since_last_purchase",
        F.when(F.col("last_purchase_date").isNull(),
            F.datediff(F.current_date(), F.col("last_purchase_date")))
    )
    .withColumn(
        "customer_value_segment",
        F.when(F.col("lifetime_sales")>=100000, "PLATINUM")
        .when(F.col("lifetime_sales")>=50000, "GOLD")
        .when(F.col("lifetime_sales")>=10000, "SILVER")
        .otherwise("BRONZE")
    )
    .withColumn(
        "customer_activity_status",
        F.when(F.col("days_since_last_purchase").isNull(), "NO_PURCHASE")
        .when(F.col("days_since_last_purchase")<=30, "ACTIVE")
        .when(F.col("days_since_last_purchase")<=90, "AT_RISK")
        .otherwise("DORMANT")
    )
    .withColumn("gold_load_timestamp", F.current_timestamp())
)
# 15. PRODUCT PERFORMANCE - grain: one row per product
df_product_metrics = (
    df_sales.groupBy("product_id").agg(
        F.countDistinct("transaction_id").alias("total_transactions"),
        F.sum("quantity").alias("total_units_sold"),
        F.sum("net_sales").cast("decimal(18,2)").alias("total_sales"),
        F.sum("discount_amount").cast("decimal(18,2)").alias("total_discount")
    )
)
df_product_performance = (
    df_gold_product.join(df_product_metrics, "product_id", "left")
    .withColumn("total_transactions", F.coalesce(F.col("total_transactions"), F.lit(0)))
    .withColumn("total_units_sold", F.coalesce(F.col("total_units_sold"), F.lit(0)))
    .withColumn("total_sales", F.coalesce(F.col("total_sales"), F.lit(0)).cast("decimal(18,2)"))
    .withColumn(
        "average_selling_price",
        F.when(F.col("total_units_sold")>0,
            F.col("total_sales")/F.col("total_units_sold"))
        .otherwise(0).cast("decimal(18,2)")
    )
    .withColumn(
        "product_performance",
        F.when(F.col("total_sales")>=100000, "TOP_PERFORMER")
        .when(F.col("total_sales")>=50000, "STRONG")
        .when(F.col("total_sales")>=10000, "NORMAL")
        .otherwise("LOW_PERFORMER")
    )
    .withColumn("gold_load_timestamp", F.current_timestamp())
)

```

```

)
# 16. WRITE GOLD DELTA TABLES
gold_dataframes = {
    "dim_date": df_date,
    "dim_customer": df_gold_customer,
    "dim_product": df_gold_product,
    "fact_daily_sales": df_daily_sales,
    "fact_product_sales": df_product_sales,
    "fact_customer_sales": df_customer_sales,
    "fact_store_sales": df_gold_store_sales,
    "fact_daily_inventory": df_daily_inventory,
    "fact_payment_summary": df_payment_summary,
    "fact_monthly_sales": df_monthly_sales,
    "customer_360": df_customer_360,
    "product_performance": df_product_performance
}
for table_name, dataframe in gold_dataframes.items():
    target = f"{GOLD}.{table_name}"
    (dataframe.write.format("delta")
     .mode("overwrite")
     .option("overwriteSchema", "true")
     .saveAsTable(target))
    print(f"Loaded: {target}")
# 17. DATA QUALITY CHECKS
spark.sql(f"""
SELECT COUNT(*) AS records,
       SUM(CASE WHEN transaction_date IS NULL THEN 1 ELSE 0 END) AS null_dates,
       SUM(CASE WHEN net_sales < 0 THEN 1 ELSE 0 END) AS negative_sales
FROM {GOLD}.fact_daily_sales
""").show()
spark.sql(f"""
SELECT COUNT(*) AS records,
       SUM(CASE WHEN available_qty < 0 THEN 1 ELSE 0 END) AS negative_inventory,
       SUM(CASE WHEN reorder_required = 1 THEN 1 ELSE 0 END) AS reorder_required
FROM {GOLD}.fact_daily_inventory
""").show()
spark.sql(f"""
SELECT COUNT(*) AS customers,
       SUM(CASE WHEN lifetime_sales < 0 THEN 1 ELSE 0 END) AS negative_lifetime_sales
FROM {GOLD}.customer_360
""").show()
# 18. OPTIMIZE
for table_name in gold_dataframes.keys():
    spark.sql(f"OPTIMIZE {GOLD}.{table_name}")
# 19. FINAL VALIDATION
spark.sql(f"SHOW TABLES IN {GOLD}").show(truncate=False)
print("GOLD LAYER PIPELINE COMPLETED SUCCESSFULLY")

```

Production Recommendations

- Use incremental processing and Delta MERGE instead of full overwrite for production workloads.
- Parameterize customer-value and product-performance thresholds as business configuration.
- Partition large time-series Gold facts by an appropriate date column where justified by workload.
- Add row-count, null, duplicate, referential-integrity and reconciliation checks to the orchestration layer.
- Expose Gold tables through Databricks SQL and BI tools such as Power BI for dashboards.